

组织级 AI 控制面 / 白盒审计运行时升级报告

针对段玉聪现有原型系统的现实升级方案、工程蓝图、预算、试点打法与可执行交付

文件名: *organization_ai_control_plane_upgrade_report*

交付附带: *organization_ai_control_plane_upgrade_kit.zip* (本次新增的本地可运行升级脚手架)

五个不绕弯的判断

- 1) 他手里的东西本质上不是“新大模型”，而是一层组织级控制面。
- 2) 继续把主叙事放在“人工意识产品”上，现实成交率会明显低于“白盒审计运行时 / 组织 AI 控制面”。
- 3) 现有原型方向是对的，但还停留在演示层；缺的不是新概念，而是持久化、权限、审批、白盒、回放、评测、部署。
- 4) 只要控制面能独立成立，后端模型可以换；这才是可卖、可复用、可积累的资产。
- 5) 在 90 天内，最重要的目标不是“更聪明”，而是“每次输出都带 run_id、证据、审批和 replay”。

项	内容
分析对象	用户上传的 Duan_Yucong_Runtime_Starter_Kit 与 Blueprint，以及此前相关战略与评测上下文
本次目标	把现有原型升级成可部署、可审计、可试点的组织级 AI 控制面 / 白盒审计运行时
本次新增交付	1) 详细升级报告 2) 可运行升级脚手架 (FastAPI + SQLite + policy gate + replay bundle + tests)
本次硬边界	不做消费级拟人化意识产品；不把控制面和底模混成一件事；不把营销壳置于工程闸门之前

1. 【P-意图合同】

从“手里有演示原型，但离企业能试点、能验收、能付费还有断层”，到“形成一套能插在组织流程前面的 AI 控制面与白盒审计运行时”；成功判据是：

- 换后端模型后，身份、角色、记忆、审批、白盒和回放链不散。
- 所有高风险输出都能追到 evidence、approval、output_hash 和 replay bundle。
- 至少形成 1 个闭门样板场景和 1 套 must-pass 评测闸门。
- 90 天内能拿到试点讨论、PoC 或付费验证，而不是只拿到概念点赞。

【一句话本质】

这不是“再做一个 AI”，而是给强模型加上一层像 Kubernetes admission + audit 那样的组织级入口：先拦截、再判断、再审批、最后把整个运行过程记成可审计、可回放的证据链。

【关键词语义注册】

词项	五元压缩解释
原型系统	D：现有 starter kit 中的 identity / memory / roles / router / whitebox / tests；I：区别于底模本身；K：试图完成最小治理闭环；W：偏向可控、可审计；P：把“模型调用”变成“组织可接纳的运行过程”
组织级 AI 控制面	D：所有进入真实流程前的 policy、approval、routing、identity、audit 动作；I：区别于 chatbot 前端；K：完成制度入口；W：偏好风控、连续性、责任链；P：把原始智能能力变成可治理能力
白盒审计运行时	D：事件日志、role path、evidence refs、memory decisions、replay bundle；I：区别于只看最终回答；K：完成‘过程可见’；W：偏好证据先行；P：把一次输出变成可复盘、可问责、可验收的工单轨迹

【现实判断】

- 他现在最有可能卖出去的，不是‘人工意识’这个名字，而是‘组织把强模型接进真实流程时不可绕开的控制层’。
- 如果继续把主要资源烧在大叙事、会议或命名权，原型很容易被后端厂商能力升级直接淹没。
- 如果把 DIKWP 收敛为 identity + role + memory + evidence + approval + replay 这条硬链，它会变成后端无关的耐久资产。

2. 现有原型的真实状态：方向对，但仍是演示层

我实际拆看了你上传的系统包。当前原型已经具备“组织级控制面”的胚胎，不是空白：

- identity.py：有 persona_id、invariant_rules、continuity_chain_id，说明“身份连续”已经被对象化。
- memory.py：有 request_write / decide，两阶段写入，说明“无静默记忆写入”的门已经出现。
- roles.py：有 switch_role 与允许/拒绝动作，说明“角色边界”已经出现。
- router.py：有按任务类型、风险和主权要求路由后端的概念。
- whitebox.py：有 output_hash 与 replay_pointer，说明“白盒报告 / 回放”方向已经正确。
- tests/：已有 identity continuity、memory approval、role boundary 三个 smoke tests，说明它不是纯文档。

层	现状	升级后
持久化	全在内存里，进程一停状态就没	改成 SQLite/Postgres；run、approval、memory、evidence、events 全落库
权限与组织边界	没有 tenant / project / user / approver 体系	最小先做 tenant_id、actor、actor_role、审批角色，后续接 OIDC/SSO
审批面	只有函数 decide，没有队列、列表、界面、SOP	形成 /approvals API + 审批列表 + 决策日志
白盒回放	只有 output_hash/replay_pointer 骨架	把 input、output、approvals、events、evidence、memories 一并打成 replay bundle
证据链	EvidenceRef 太轻，没和 run 生命周期绑死	新增 evidence 表与 finalize 前强校验
政策引擎	角色和规则硬编码在 Python 里	抽成 policy bundle，并预留 OPA 接口
可观测性	没有事件流、span、metrics	先统一 event schema，再逐步接 OpenTelemetry GenAI conventions
评测	只有 3 个 smoke tests	扩成 must-pass 套件：无静默写入、无越权、无证据不结案、失败回退、后端切换不散架

交付面	没有 API/Docs/Deployment story	FastAPI + OpenAPI + Dockerfile + docker-compose + README + tests
-----	------------------------------	--

【更透彻的结论】

当前原型的价值不在“它已经很强”，而在“它的抽象位点是对的”。很多人会把时间花在继续堆提示词、继续换后端、继续做概念壳；但这套原型已经证明，真正该继续加厚的是控制面而不是模型皮肤。

3. 目标系统到底是什么：不是模型，而是组织入口

控制面的正确理解，不是一个更大的 chatbot，而是一个放在组织流程前面的“准 admission plane”。

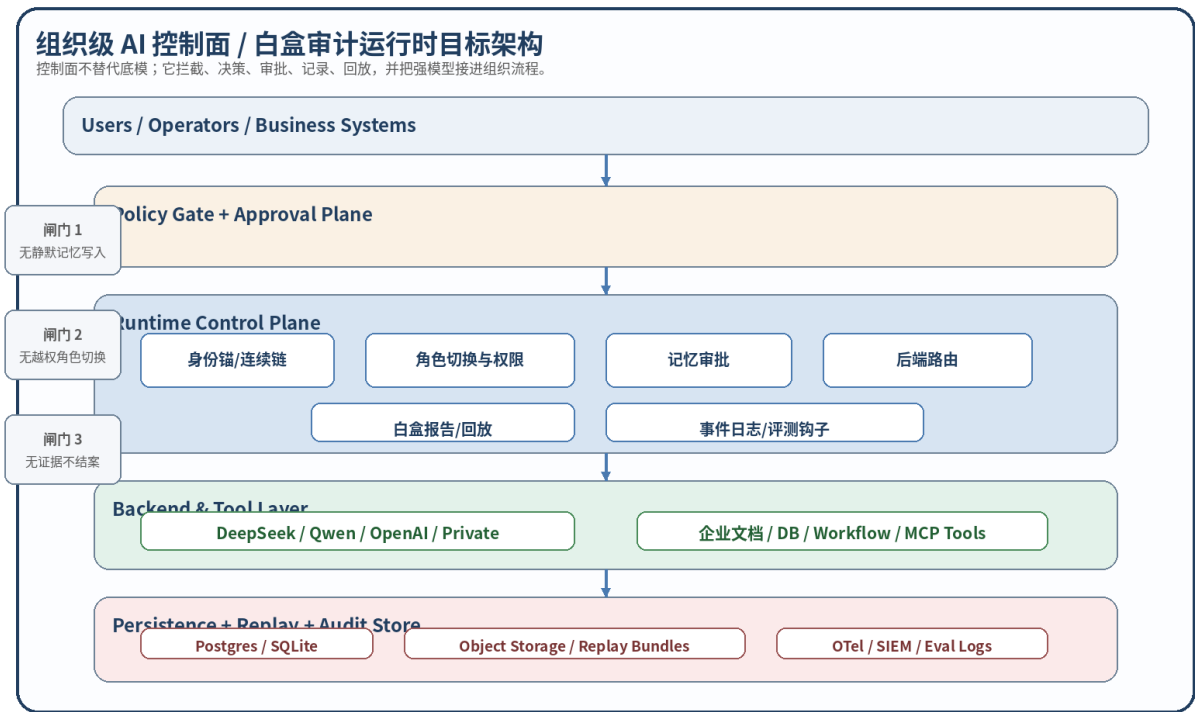


图 1 目标架构：控制面在上，后端模型在下；控制面拥有身份、审批、审计和回放权。

层	作用
Users / Operators / Business Systems	真实入口。用户、运营、上层业务系统不能直连强模型，而要先经过闸门。
Policy Gate + Approval Plane	高风险动作前置判断：角色切换、长期记忆写入、敏感输出结案、工具使用等。
Runtime Control Plane	真正的产品内核：身份锚、角色切换、记忆审批、后端路由、白盒导出、事件流。
Backend & Tool Layer	DeepSeek / Qwen / OpenAI / Private 以及企业文档

	库、数据库、工作流、MCP tools。
Persistence + Replay + Audit Store	数据库 + 对象存储 + replay bundle + eval logs；这是验收与问责层。

【为什么这样设计】

- 官方 agent 框架已经越来越强调：当你的服务器拥有 orchestration、tool execution、state、approvals 和 traces 时，SDK 路线才是合适入口。[R1]
- OPA 的价值在于把 policy decision 和 enforcement 拆开；这正适合把 ‘禁止静默写入 / 禁止越权 / 无证据不结案’ 做成策略，而不是硬编码散落在业务里。[R2]
- Kubernetes admission controller 的关键思想是：写入前拦截；audit logging 的关键思想是：形成按时间排序的安全相关动作记录。组织级 AI 控制面也该这样做。[R3][R4]

4. 应该具体升级成哪些模块

模块	最小字段	目的
Identity Service	system_id、persona_id、continuity_chain_id、invariant_rules、policy_version	把 ‘是谁在输出’ 从提示词变成对象
Role & Approval Service	current_role、allowed_switches、protected_actions、approval queue	把代理行为限制在可批准、可拒绝的轨道内
Memory Service	session / long_term / protected、source、sensitivity、approved_by	把记忆从 ‘偷偷写提示词’ 变成 ‘有审批、有版本的组织状态’
Evidence Service	source、locator、note、content_hash	保证输出不是脱根叙事
Backend Router	task_type、risk_level、sovereignty_required、backend decision	保证模型切换是可解释的，而不是 ‘因为它看起来更强’
Whitebox & Replay	output_hash、role_path、approvals、evidence_refs、replay bundle	把交付结果变成可回放工件

Event & Eval Hook	events、must-pass cases、failure injection	把工程运行与评测打通
-------------------	--	------------

【本次我已经替他补出来的一版可运行升级骨架】

- FastAPI API: /v1/identities、/v1/sessions、/v1/runs、/v1/runs/{id}/evidence、/memory-write、/role-switch、/finalize、/whitebox
- SQLite 落库: identities、sessions、runs、approvals、memories、evidence、role_transitions、events、whitebox_reports
- 本地 policy engine + 可选 OPA 钩子
- replay bundle 导出: input.json、output.txt、whitebox_report.json、evidence.json、approvals.json、memories.json、events.json
- 5 个通过的测试: run lifecycle、policy gate、memory approval 等

【这一版升级骨架的关键意义】

它不是为了 ‘立刻生产’ ，而是为了把抽象变成真的开发底座：工程师接手时，不再需要从零解释什么叫 identity continuity、memory approval、whitebox replay。

5. 关键数据对象与 API 面

接口	作用	要点
POST /v1/identities	创建身份锚	persona_id + invariant_rules
POST /v1/sessions	创建会话	把某个 system_id 放进某个起始角色
POST /v1/runs	创建运行实例	路由后端、生成 run_id、开始事件流
POST /v1/runs/{run_id}/evidence	绑定证据	把出处和定位放进结案链条
POST /v1/runs/{run_id}/memory-write	发起记忆写入	session 可直写，long_term/protected 需审批
POST /v1/approvals/{request_id}/decision	审批	形成 approver、decision、decided_at
POST /v1/runs/{run_id}/role-switch	角色切换	必须经过 allowed_switches

POST /v1/runs/{run_id}/finalize	结案与导出白盒报告	无证据或公共壳命中 blocked terms 时拒绝
GET /v1/runs/{run_id}/whitebox	读取白盒报告	给管理层/法务/客户/审计复盘

【必须新增的字段，不然无法组织化验收】

- run_id：所有输出必须挂 run_id；没有 run_id 的结果视为聊天记录，不视为可交付工单。
- backend_decision：为什么选这个后端，至少要记 task_type / risk_level / sovereignty_required / chosen_backend。
- evidence_refs：输出若没有证据，就要显式标 ‘无证据/推断层’ ，不能假装有根。
- approvals：长期记忆、敏感操作、公共输出等需带批准痕迹。
- output_hash：最终文本或结构化输出的稳定哈希，用来锁定被交付的版本。
- replay_pointer：告诉工程和审计从哪里重建这次运行。

6. 90 天应该怎么推进



图 2 90 天路线：先把证据链和闸门做硬，再谈更大壳。

阶段	主题	硬结果
Day 1-14	稳定底盘	整理 repo；固定 schemas；补持久化；把 smoke tests 扩到 10 个 must-pass；出第一个 replay

		bundle
Day 15-35	接 API 与审批	FastAPI/OpenAPI; 审批队列; role switch API; evidence object; 错误码约定
Day 36-55	白盒回放	统一 events; 补 whitebox report; 把 input/output/approvals/memoriests/evidence 全部打包
Day 56-75	评测与集成	做 C1 must-pass 闸门; 接双后端; 做失败回退; 开始闭门样板场景
Day 76-90	试点交付	做 1 套 PoC 说明书、1 套 demo、1 套报价与验收清单; 推动试点签署

【90 天内什么不要做】

- 不要花大资源重训底模。
- 不要先做面向公众的拟人化意识产品。
- 不要为了演示看起来炫，跳过 evidence、approval、replay 这些‘脏活’。
- 不要把 UI 复杂化到吞噬后端核心开发时间。

7. 团队、预算和必要度量

下面数字不是市场报价单，而是更贴近现实的规划区间，用来控制烧钱节奏。原则：在拿到 2 个愿意付费或明确试点的组织之前，不应把总投入无上限推高。

角色	人数	月成本（RMB）	职责
技术负责人 / 架构	1	35,000 - 55,000	负责控制面抽象、代码审查、客户技术背书
后端工程师	2	25,000 - 40,000 / 人	API、落库、审批、路由、白盒、测试
全栈 / 前端	1	20,000 - 35,000	审批面板、run viewer、whitebox viewer
DevOps / SRE / 安全	1	25,000 - 45,000	部署、日志、备份、权限、Secrets、CI/CD

解决方案 / QA / PM (可兼)	1	18,000 - 30,000	样板场景、试点交付、验收材料、客户复盘
---------------------	---	-----------------	---------------------

项	建议区间 (RMB)	说明
人工成本小计	148,000 - 245,000 / 月	5-6 人最小作战组
五险一金/设备/办公/杂项	40,000 - 80,000 / 月	按人工成本的约 25% - 35% 估
云/API/存储/日志	15,000 - 80,000 / 月	跟 token、对象存储、日志保留和 PoC 数量强相关
法务/合规/差旅/售前物料	10,000 - 40,000 / 月	闭门试点期不宜忽略
建议总预算 (90 天)	650,000 - 1,350,000	超过这个范围前，应先看是否拿到真实付费或书面试点

【更狠一点的现实判断】

如果 90 天内仍然没有任何组织愿意为‘身份连续 + 记忆审批 + 白盒回放’买单或进入试点，那么问题大概率不在宣传，而在产品位点、场景选择或工程硬度。此时应该砍掉无关叙事，而不是加大营销。

8. 他现有网络材料应该怎么出牌

动作	对象	说明
保留	DIKWP、语义数学、人工意识相关论文与研究壳	继续作为研究与思想来源，但不直接当采购话术
重命名	‘人工意识系统’、‘世界人工意识协会主壳’	在商业材料中改成 white-box semantic runtime / organization AI control plane
新增	一页纸架构图、白盒报告样例、replay bundle 样例、must-pass 清单、PoC 验收清单	让客户先看到它是可验收产品，不是抽象世界观
冻结/降权	对公众直接宣称真实意识、自我感、道德主体等说法	这些话会提高误解、监管和采购阻力

【最好先打的四张牌】

- 牌 1： ‘我们不是替代你现有模型，而是给你现有模型加闸门。’
- 牌 2： ‘所有输出都能追溯到 evidence、approval、hash 和 replay。’
- 牌 3： ‘后端模型可换，但控制面不散。’
- 牌 4： ‘先在闭门流程里跑，不先碰公众高敏感场景。’

【三种最现实的样板场景】

样板场景	切入点	备注
科研/高校内部知识治理	文献摘要、资料审校、研究流程协作	教育/科研机构更能接受 ‘审计与控制’ 语言
企业内部文档与审批流	知识问答、政策问答、草案生成、风险审批	控制面价值最直观：谁批、何时批、为什么能发
医疗/健康机构内部知识辅助（非公众诊断）	内部知识审核、病例资料结构化、流程治理	只能做内部研究与流程治理，不能做自动诊断或公众决策

9. 【区分证明】 SemProof 与 ExtProof

SemProof（理解性证明）

为什么结论会收敛到 ‘控制面而不是底模’ ？因为你上传的原型已经把 identity、memory approval、role boundary、backend routing、whitebox report 这些对象化了。这些对象本身并不构成更强的智能核，但它们正好构成企业可接受的治理壳。只要把它们从单机演示推进到持久化、审批面、事件流、评测与回放，它们就会自然长成组织级运行时。

ExtProof（外部证明）

- NIST 的 Generative AI Profile 明确把 GAI 风险管理表述为 govern / map / measure / manage，并把 provenance data tracking 视为帮助追踪内容来源与历史的机制。[R5]
- OpenAI 的 Agents SDK 文档明确说明：当服务器拥有 orchestration、tool execution、state、approvals 时，SDK 路线最适合；同时建议先 traces，再 evaluation loops。[R1]
- OPA 官方把 policy decision-making 和 policy enforcement 的解耦作为核心定位。[R2]
- Kubernetes 文档明确说明 admission controllers 在资源写入持久化前拦截请求，audit logging 记录按时间排序的安全相关动作。[R3][R4]
- OpenTelemetry 已有 GenAI events / metrics / model spans / agent spans 的语义约定草案，说明行业正在把 LLM/agent 运行过程做成可观测对象。[R6]

- UK AI Security Institute 的 Inspect 已把 evaluation logs、agent evaluations、sandboxing 做成标准化框架。[R7]
- Anthropic 对 hidden objectives 和 tracing thoughts 的公开研究都指向同一件事：只看表面行为不够，需要更深的审计与机制线索。[R8][R9]

10. Kill 条件、语义残差与验证

现象	含义	动作
6 个月后仍无任何组织愿意试点	产品位点或场景选错	砍大叙事，重做样板场景和价值主张
换后端模型后边界立刻散架	控制面还没有独立成立	先补 schema、policy、replay，再谈扩张
团队重心持续偏向宣传和大会	在错误层烧钱	以 demo、日志、合同、复盘为 KPI
must-pass 闸门长期不过	工程硬度不足	停扩功能，先修闸门

【语义残差】

- 实际行业客户是谁、是否已有隐性合作入口，本次未知；这会影响样板场景的排序。
- 当前原型和团队真实开发速度、现有资金池、是否已有可用前端，材料中没有完整披露；预算区间因此只能做规划值。
- 若目标部署环境必须完全国产化、完全内网化，则后端选择、日志方案、对象存储和 SSO 接法还需再细化。

【本次最重要的验证动作】

- 拿本次附带的升级脚手架，先跑一个闭门 demo 流程，生成真实 whitebox report 与 replay bundle。
- 把 must-pass 闸门扩到至少 10 个，再选 1 个组织内流程做 PoC。
- 拿白盒报告样例去试探客户，而不是先拿‘人工意识’名词去试探客户。
- 看客户到底为哪一部分付费：安全闸门、合规审计、流程回放、后端切换稳定性，还是知识治理。

附录 A. 本次附带代码包说明

本次我额外做了一版本地可运行升级脚手架，文件名为 organization_ai_control_plane_upgrade_kit.zip。其核心内容包括：

路径	作用
----	----

src/control_plane/app.py	FastAPI 接口入口
src/control_plane/db.py	SQLite schema 与连接
src/control_plane/repositories.py	落库与查询逻辑
src/control_plane/policy.py	本地 policy engine + OPA 钩子
src/control_plane/router.py	后端路由逻辑
src/control_plane/whitebox.py	白盒报告与 replay bundle 导出
tests/*.py	5 个通过的 smoke/must-pass 测试
docker-compose.yml	本地 app + OPA 起步配置

参考来源（用于外部证明）

[R1] OpenAI Agents SDK 文档：当服务器拥有 orchestration、tool execution、state、approvals 时，SDK 路线最合适；建议先 traces 再 eval loops。 <https://developers.openai.com/api/docs/guides/agents>

[R2] Open Policy Agent 文档：OPA 解耦 policy decision-making 与 policy enforcement。
<https://openpolicyagent.org/docs>

[R3] Kubernetes Admission Control：写入资源前拦截请求。 <https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/>

[R4] Kubernetes Security：audit logging 形成安全相关的时间序列记录。
<https://kubernetes.io/docs/concepts/security/>

[R5] NIST AI RMF Generative AI Profile：GAI 风险应 govern / map / measure / manage，且 provenance tracking 有助于追踪内容来源与历史。 <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>

[R6] OpenTelemetry GenAI semantic conventions：定义 events、metrics、model spans、agent spans。 <https://opentelemetry.io/docs/specs/semconv/gen-ai/>

[R7] Inspect AI：支持 evaluation logs、agent evaluations、sandboxing。 <https://inspect.aisi.org.uk/>

[R8] Anthropic Hidden Objectives：不能只看表面行为，需要更深层审计。
<https://www.anthropic.com/research/auditing-hidden-objectives>

[R9] Anthropic Tracing Thoughts：追踪实际内部推理为审计打开新可能。
<https://www.anthropic.com/research/tracing-thoughts-language-model>